Chapter 1. Introduction

> *Concurrency is about dealing with lots of things at once. Parallelism is about doing lots of things at once.*
>
> —Rob Pike

To truly appreciate something, it is crucial to know how it came to be, especially if we can discern the steps taken and the challenges overcome along the way. This understanding not only highlights ongoing progress but also helps us understand its relevance. Similarly, Java concurrency has come a long way since its inception. It took a long time to evolve to its current state. But if we want to understand recent advancements, such as virtual threads and structured concurrency in modern Java, we must first delve into its evolution. In this chapter, we will give you an initial view of Java concurrency and then briefly discuss how it has developed over time.

A Brief History of Threads in Java

Java was designed with concurrency in mind; it was one of the first languages to provide built-in support for multithreading. Over the years, Java's concurrency capabilities have been improved and refined, leaving behind some potholes and lessons along the way.

Java concurrency began with basic synchronization and thread management. Then came the introduction of the java.util.concurrent package in Java 5, which brought important new capabilities such as the Executor framework, locks, and concurrent collections. Next, with the introduction of the Fork/Join framework in Java 7, Java engaged the concurrency performance of multicore processors. And most recently, with Project Loom, Java addressed the complexity and limitations of traditional thread-based concurrency, with lightweight, user-mode threads and structured concurrency, ultimately aiming to make concurrent development simpler and more efficient.

Why does this evolution matter so much? As we uncover how Java's concurrency story unfolds, we discover a relentless pursuit of greater efficiency and simplified programming in the face of ever-growing complexi-

ty. This narrative extends beyond just Java; it reflects the trajectory of software development and Java's continued aspirations.

Let's dive deeper into understanding this evolution and appreciate the strides made in Java concurrency.

Java Is Made of Threads

Concurrency was an explicit design goal of Java, a language that was released with built-in thread capability and a threading feature that was a key differentiator of the language. It removed developers' dependency on operating-system-specific features to achieve concurrency.

In Java, a thread is the smallest unit of execution. It is an independent path of execution running within a program. Threads share the same address space, meaning they have access to the same variables and data structures of the program. However, each thread maintains its own program counter, stack, and local variables, enabling it to operate independently. This design also facilitates interaction among threads when necessary.

However, Java's threading model relies on the underlying operating system to schedule and execute threads. The operating system allocates CPU time to each thread, managing the transition between threads to ensure efficient execution. By distributing threads across multiple CPUs, the system can achieve true parallelism, enhancing the performance of concurrent applications (Figure 1-1).

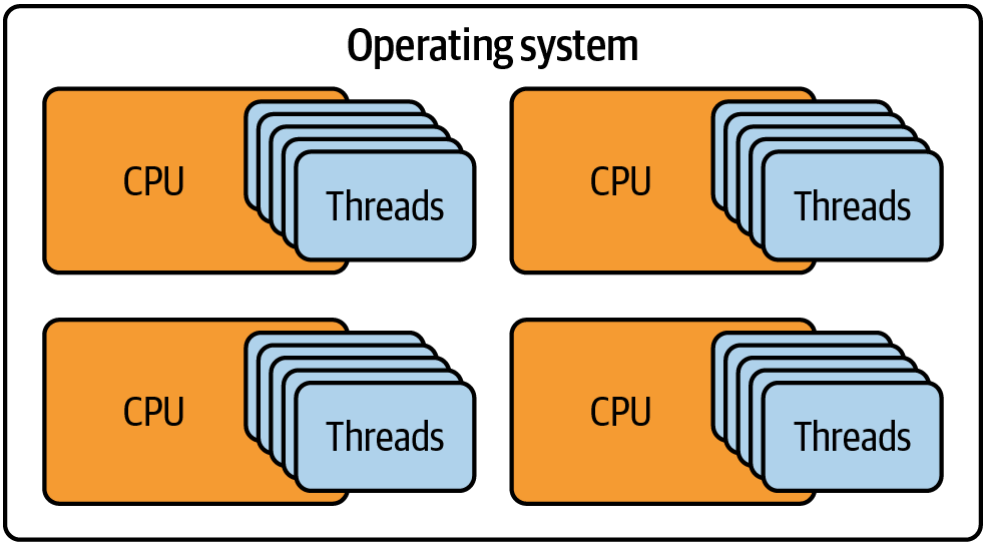

Figure 1-1. Execution of threads by different CPUs

The more threads we can have in a Java program, the more execution environment we effectively create. This gives us the ability to execute several operations simultaneously. This is particularly beneficial for applications that require high levels of parallelism or concurrency, such as

web servers, data processing pipelines, and real-time systems. By leveraging multiple threads, these applications can perform multiple tasks simultaneously, thereby improving throughput and responsiveness.

PARALLELISM VERSUS CONCURRENCY

The terms *parallelism* and *concurrency* are often used interchangeably. However, they mean two different things. Parallelism entails doing more than one thing simultaneously, so multiple processing units, such as two or more CPU cores, are needed. Concurrency is about designing programs where portions of their operation can overlap—even if not always simultaneously. Think of parallelism as multiple workers building a house side-by-side, while concurrency is like a single chef juggling multiple dishes in the kitchen. Now, if we add more chefs to the kitchen, the overall output improves, but it is not necessarily parallel; one chef might be chopping onions while another is putting a dish in the oven. The end goal remains preparing the meal—perhaps even multiple meals.

The notion of threads forms the very fabric of how the entire Java ecosystem is implemented and how it functions. It's the bedrock on which lots of powerful features and tools are based, and functions that many programmers take for granted simply wouldn't be possible without it. Whether it's the garbage collection system, which tackles the problem of memory management in Java, or the process of executing the simple output of a "Hello, World!" program in Java, threads are working away in the background.

Here's an example. Anyone who has written a Java program, even for the first time, will be familiar with the line most programming books start out with. It may seem like a single-threaded operation; however, the Java Virtual Machine (JVM) actually executes this code in a thread, commonly referred to as the *main thread*:

```
public class HelloWorld {
  public static void main(String[] args) {
    System.out.println("Hello, World!");
        // Displaying the thread that's executing the main method
    System.out.println("Executed by thread: "
        + Thread.currentThread().getName());
  }
}
```

When we run this program, the output would include the name of the thread executing the main method, which is typically `main`:

```
Hello, World!
```

```
Executed by thread: main
```

This example shows that even the most basic Java programs are already fundamentally threaded. The implication is profound: we, as Java developers, are harnessing the power of threads, whether we realize it or not, in virtually every part of our job, from running the most straightforward programs to using even the most advanced techniques, such as garbage collection.

And so, threads are a required element of Java—they're part of what makes Java a language that can scale up to handle large databases and massive distributed systems.

Threads: The Backbone of the Java Platform

Java threads are integral to all layers of the Java platform, playing a crucial role in various aspects beyond just executing code. For instance, they're the basis of exception handling, debugging, and profiling Java applications.

Exceptions and threads

In Java, every thread has its own separate call stack that records all method call invocations done during the thread's lifetime. When an exception is raised, that thread's call stack becomes a vital part of the diagnostic history; it shows the sequence of method invocations that led to the exception, helping developers trace the root cause of an issue. Here's that same example again:

```
import java.sql.SQLException;

public class CallStackDemo {
  public static void main(String[] args) throws InterruptedException {
    Thread thread = new Thread(CallStackDemo::processOrder);
    thread.setName("mcj-thread");
    thread.start();
    thread.join();
  }

  static void processOrder() {
    validateOrderDetails();
  }

  static void validateOrderDetails() {
    checkInventory();
  }

  static void checkInventory() {
```

```
        updateDatabase();
    }

    static void updateDatabase() {
        try {
            throw new SQLException("Database connection error");
        } catch (SQLException e) {
            throw new InventoryUpdateException("Database Error: " +
                "Unable to update inventory", e);
        }
    }
}

class InventoryUpdateException extends RuntimeException {
    public InventoryUpdateException(String message, Throwable cause) {
        super(message, cause);
    }
}
```

In this scenario, the database update operation fails, which propagates back up the call stack. The main thread starts calling `processOrder`, which calls `validateOrderDetails`, which calls `checkInventory`, which calls `updateDatabase`, which then throws the exception. We now will get the following output in the console:

```
Exception in thread "mcj-thread" ca.bazlur.mcj.chap1.InventoryUpdateExcept
Database Error: Unable to update inventory
    at ca.bazlur.mcj.chap1.CallStackDemo.updateDatabase(CallStackDemo.java
    at ca.bazlur.mcj.chap1.CallStackDemo.checkInventory(CallStackDemo.java
    at ca.bazlur.mcj.chap1.CallStackDemo.
    validateOrderDetails(CallStackDemo.java:22)
    at ca.bazlur.mcj.chap1.CallStackDemo.processOrder(CallStackDemo.java:1
    at java.base/java.lang.Thread.run(Thread.java:1583)
Caused by: java.sql.SQLException: Database connection error
    at ca.bazlur.mcj.chap1.CallStackDemo.updateDatabase(CallStackDemo.java
    ... 4 more
```

This trace allows developers to inspect the call stack within a specific thread context (i.e., the context of the `mcj-thread` thread where the exception occurred). This granularity is highly beneficial in the JVM, especially for debugging multithreaded applications where several threads may run concurrently on the same or different tasks. This detail helps us pinpoint issues faster, making debugging more focused and efficient, and leading to a quicker resolution.

Debugger and threads

How does the Java debugger figure out where exactly to pause the execution of a running application? The answer again is threading. By attaching itself to the various threads in the application, the debugger can select which of these individual threads to examine or even change their state while debugging an active application. This is fundamental to your ability to find and fix bugs in applications where several threads might be simultaneously executing at different stages, and where different threads might be interacting in ways that need to be understood.

Each action can be “stepped into,” “stepped over,” or “stepped out of” with respect to a thread. One or more independent call stacks for the active threads may be inspected in a debugging session. The call stack tells you what happened before, such as what caused a thread to be at a given point in a program. Understanding how a thread got to a certain state is invaluable when you’re trying to figure out the cause of an unexpected exception or an erroneous data manipulation.

Profiler and threads

Threads are equally vital to understanding Java’s operational dynamics, and they play a crucial role in profiling. Just as threading facilitates pinpoint precision in debugging, its utility extends to offering a granular perspective in profiling practices. In fact, threads are the backbone of performance analysis in Java. Profiling tools rely on threads to give you a detailed look at how your application is running. They use thread information to pinpoint slowdowns, troubleshoot tricky timing issues in multithreaded code, and help you find ways to make your application run faster. In short, threads are the key to understanding and improving the performance of your multithreaded Java applications.

Java threads play a significant role in many operations, such as diagnosis, debugging, and profiling, by providing detailed insight into the execution of individual thread-level programs. Despite not being directly used by developers, they keep operating underneath and augment the benefits of Java applications as well. Examples of this are garbage collection threads used for the management of memory; compiler threads in the just-in-time (JIT) system for performance enhancement; signal dispatcher, finalizer, and reference handler threads for smooth execution of JVM; and virtual machine (VM) and service threads for the core JVM tasks and diagnostics. Hence, it is easy to say that Java threads are one of the most essential layers of the Java platform, and a Java developer needs to understand them.

The Genesis of Java 1.0 Threads

Java 1.0 was released in 1996 and came with built-in support for threads. This defining feature set it apart from many languages at that time. In the early days, you could create threads by either extending the `Thread` class or implementing the `Runnable` interface. For example:

```
public class ThreadCreationDemo {
  public static void main(String[] args) {
    // Extending Thread class
    Thread t1 = new ThreadByExtension("Worker-1"); ❶
    t1.start();
    // Implementing Runnable (classic)
    Thread t2 = new Thread(
        new RunnableImplementation(), "Worker-2"); ❷
    t2.start();
    // Anonymous inner class (Java 1.1)
    Thread t3 = new Thread(new Runnable() { ❸
      @Override
      public void run() {
        System.out.println("Anonymous: " +
            Thread.currentThread().getName());
      }
    }, "Worker-3");
    t3.start();
    // Lambda expression (Java 8)
    Thread t4 = new Thread(() -> ❹
        System.out.println("Lambda: " +
            Thread.currentThread().getName()),
        "Worker-4");
    t4.start();
  }
}

class ThreadByExtension extends Thread {
  public ThreadByExtension(String name) {
    super(name);
  }
  @Override
  public void run() {
    System.out.println("Extended Thread: " + getName());
  }
}

class RunnableImplementation implements Runnable {
  @Override
  public void run() {
    System.out.println("Runnable: " +
        Thread.currentThread().getName());
```

```
    }
}
```

Key design considerations from the start:

1. Thread extension offers direct access to thread methods but uses up your single inheritance slot.
2. The `Runnable` implementation became the preferred approach, separating task definition from thread management and preserving inheritance flexibility.
3. Anonymous inner classes (Java 1.1) reduced boilerplate for one-off tasks before lambdas existed.
4. Lambda expressions (Java 8) made the `Runnable` implementation even more concise, since it's a functional interface.

This foundational API has remained unchanged for nearly three decades, demonstrating its robust design. While Java has added countless concurrency enhancements, from `java.util.concurrent` to virtual threads, these original building blocks remain the foundation of Java's threading model.

Starting Threads

Whichever method we choose for creating a thread begins by invoking the `start` method on the `Thread` object. This is an essential step because calling `start` does more than just execute the `run` method; it also performs setup tasks like allocating system resources. The `start` method subsequently calls the `run` method in a new thread of execution.

NOTE

It's crucial not to call the `run` method directly. Doing so will execute the `run` method in the calling thread, not in a new thread.

However, in modern Java applications, using the Executor framework is more efficient than manually creating threads through constructors. Executors abstract the process of thread management by maintaining a thread pool—a group of pre-created threads ready for executing tasks. This method significantly reduces the overhead associated with creating new threads.

When a task is submitted to an executor, it's placed in a queue. A thread from the pool then picks up the task from the queue and executes it. This

setup simplifies concurrent programming and optimizes resource utilization by reusing threads.

Here's a simple example demonstrating how to use an executor to manage a thread pool:

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;

public class ExecutorExample {
    public static void main(String[] args) {
        try (ExecutorService executor = Executors.newFixedThreadPool(5)) {
            for (int i = 0; i < 10; i++) {
                final int taskId = i;
                executor.submit(() -> {
                    System.out.println("Executing task " + taskId
                            + " in thread " + Thread.currentThread().getNa
                });
            }
        }
    }
}
```

Understanding the Hidden Costs of Threads

Many of today's web applications run on a thread-per-request model, where a thread is assigned to each request—the thread manages the whole request/response lifecycle. For instance, let's consider the lifecycle of a request on a typical web application running under a servlet container (Apache Tomcat, Jetty, or any Java EE web server). A *servlet container* is software responsible for processing requests coming to the web application. When a request arrives at the servlet container, the container assigns the request to one of the threads in its thread pool to process the request. That thread fires up, in effect saying, "Hey, I got this request coming from the user. It's mine now; I'll take care of it." The thread essentially takes responsibility for processing the whole request/response lifecycle (Figure 1-2).

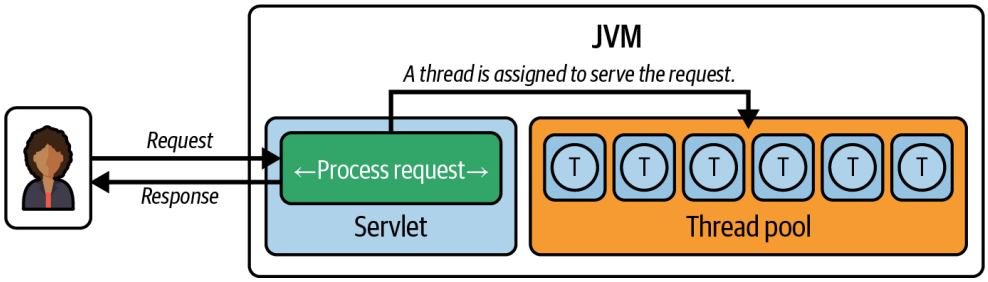

Figure 1-2. Thread pool handling servlet requests

This is particularly helpful when we have large numbers of simultaneous connections because increasing concurrency, in turn, increases the throughput of the web application. In fact, this is a key feature of the thread-per-request model, which allows modern web applications to scale in order to serve a growing volume of requests efficiently.

Amazingly, most modern operating systems can handle millions of concurrent connections, so it would seem reasonable to create yet more threads in order to improve throughput. More threads in the thread pool should indeed equate to more throughput, according to Little's Law.[1] However, even though that assumption appeared to be accurate, it turns out that this line of reasoning is a bit slippery—and will not always produce the expected results.

NOTE

Throughput in web applications is the rate at which requests are processed and the server delivers responses, typically measured in requests per second (RPS) or transactions per second (TPS). It indicates the application's capacity to handle load, serving as a critical metric for performance, scalability, and resource utilization. Throughput can be calculated as

$$\text{Throughput} = \frac{\text{Total Request Processed}}{\text{Total Time}}$$

This serves as a guide for benchmarking performance, scalability, and demand planning, and it paves the way for the optimal use of resources that ensure a quality user experience.

An important detail to bear in mind is the memory footprint of each thread: about 2 MiB[2] of memory (outside the heap per thread). This can quickly become a lot, especially in large-scale applications running thousands of concurrent connections—the aggregate memory footprint of the threads can make a world of difference in the actual number of connections you can run. Additionally, suppose your application uses all physical memory. In that case, you'll find yourself paging to disk, a significantly slower operation than even accessing RAM: the time-to-read difference when reading from disk versus RAM is a factor of 1,000. This alone can heavily impact performance.

Also, it is essential to understand that Java threads are actually a thin wrapper around the native threads provided by the host operating system. This means that the maximum number of threads you can create for any application is effectively limited by the native thread creation limit provided by the host operating system. Almost all operating systems have an upper limit on the number of threads that can be spawned. So an application's scalability potential is limited by this bottleneck.

Besides, there is the expense of context switching. Thread creation isn't just a memory overhead. It's also a CPU overhead. When you switch between threads, that's a context switch. The context of the current thread needs to be stored, and the context of the new thread needs to be brought back. All that context switching costs CPU cycles, and, on systems under high load, this overhead can make a huge difference in performance. This is another underappreciated source of weight in your application.

How Many Threads Can You Create?

Now that we have discussed the costs associated with threads, let's examine how to measure the limitation of thread creation in your environment. By running a simple test program, you can determine the maximum number of threads your system can handle before experiencing issues. Here's a skeleton code snippet to start:

```
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.locks.LockSupport;

public class ThreadLimitTest {
    public static void main(String[] args) {
        var threadCount = new AtomicInteger(0);
        try {
            while (true) {
                var thread = new Thread(() -> {
                    threadCount.incrementAndGet();
                    LockSupport.park();
                });
                thread.start();
            }
        } catch (OutOfMemoryError error) {
            System.out.println("Reached thread limit: " + threadCount);
            error.printStackTrace();
        }
    }
}
```

Let's execute this program to see how many threads can be created before running out of memory or hitting other system limitations:

```
Reached thread limit: 16363
java.lang.OutOfMemoryError: unable to create native thread: possibly out o
memory or process/resource limits reached
    at java.base/java.lang.Thread.start0(Native Method)
    at java.base/java.lang.Thread.start(Thread.java:1526)
    at ca.bazlur.chapter0.ThreadLimitTest.main(ThreadLimitTest.java:15)
```

On my machine, I encountered an `OutOfMemoryError` after successfully creating 16,363 threads. This experiment highlights the point that there's a finite limit to the number of threads one can create, a constraint that is largely dictated by the underlying operating system and the hardware. Hence, the limit can be different on your machine.

As you can see, while threads provide substantial benefits in web application scalability, they do come with associated costs. These costs, though sometimes less obvious, should be carefully considered to ensure optimal performance.

Resource Efficiency in High-Scale Applications

Today's software applications are often expected to process large amounts of data and handle high volumes of incoming traffic simultaneously. These dynamics create a significant challenge for staying fiscally sound, especially when the cloud has become the default environment in which many businesses operate. Even the slightest degree of resource inefficiency can quickly turn into escalating costs. Since we have only a finite number of threads, it's essential to use them carefully. But threads get blocked in practice, so they're often not used effectively.

Consider the following example, where a method calculates a person's credit score based on various factors:

```
public Credit calculateCredit(Long personId) {
    var person = getPerson(personId);          // Database call - blocks t
    var assets = getAssets(person);            // API call - blocks thread
    var liabilities = getLiabilities(person);  // Database call - blocks t
    importantWork();                           // CPU-intensive work
    return calculateCredits(assets, liabilities);
}
```

The preceding code snippet shows a sequence of five method invocations, each happening one after the other. Suppose each would typically take 200 milliseconds to complete. Then the time it takes for the `calculateCredit()` method to execute would be about 1 second—roughly five times as long (1,000 milliseconds = 200 milliseconds × 5).

The real inefficiency here isn't that the thread is idle, but rather that it's blocked during input/output (I/O) operations. When the thread executes `getPerson(personId)`, it makes a database call and must wait for the network response. During this waiting period, the thread cannot be reas-

signed to handle other requests or perform other work. The same blocking occurs during the `getAssets()` and `getLiabilities()` calls. While the thread is technically "busy" executing these methods, it's spending most of its time blocked on I/O operations rather than doing productive computational work.

This blocking behavior represents a significant inefficiency: expensive thread resources are tied up waiting for external systems to respond, when they could potentially be serving other requests. In high-throughput applications, this can severely limit scalability since each blocked thread represents one fewer thread available to handle incoming requests.

Dealing with the key challenge of achieving high thread efficiency, in particular, trying to minimize the time that threads spend blocked on I/O operations, has driven many of Java's evolutions over time. This includes support for various paradigms such as asynchronous method invocations (allowing methods to run independently while I/O operations complete), thread pooling (reusing a fixed number of threads to perform tasks), and, more recently, modern reactive programming models (emphasizing non-blocking, event-driven interactions).

Let's start by introducing the classical methods of threading, such as manually creating and managing individual threads, and gradually move into what modern offerings provide.

The Parallel Execution Strategy

Looking at our preceding code snippet, we can see that there are some method invocations, namely, `getAssets()`, `getLiabilities()`, and `importantWork()`, that do not have interdependencies, so these methods can actually be dispatched to run in parallel. This approach to parallel execution will reduce the duration for which our main thread is blocked, even though we haven't totally eliminated blocking.

Our parallel execution strategy effectively allows the main thread to diminish idle time. This essentially means that, since the main thread is less idle, once this `calculateCredit()` method is done executing, it can quickly be free to do other work.

Let's start by creating the basic data structures we'll need for our credit calculation system:

```
// Credit calculation models
record Credit(double score) {}
record Person(Long id, String name) {}
```

```
record Asset(String type, double value) {}
record Liability(String type, double amount) {}
```

Now, let's implement our parallel execution approach:

```
import java.util.List;
import java.util.concurrent.Callable;
import java.util.concurrent.atomic.AtomicReference;

Credit calculateCreditWithUnboundedThreads(Long personId)
                                        throws InterruptedException {

    var person = getPerson(personId);

    var assetsRef = new AtomicReference<List<Asset>>(); ❶
    var t1 = new Thread(() -> {
        var assets = getAssets(person);
        assetsRef.set(assets); ❷
    });

    var liabilitiesRef = new AtomicReference<List<Liability>>();
    Thread t2 = new Thread(() -> {
        var liabilities = getLiabilities(person);
        liabilitiesRef.set(liabilities);
    });

    var t3 = new Thread(() -> importantWork()); ❸

    t1.start(); ❹
    t2.start();
    t3.start();

    t1.join(); ❺
    t2.join();

    var credit = calculateCredits(assetsRef.get(), liabilitiesRef.get());

    t3.join(); ❼

    return credit;
}
```

Let's see how the preceding code works:

- ❶ Uses `AtomicReference` to safely share data between threads.
- ❷ Thread-safe assignment of results that can be accessed from the main thread.
- ❸ Independent work that can run in parallel without affecting the credit calculation.

❹ All three threads start executing concurrently, reducing overall execution time.

❺ Waits for assets and liabilities threads to complete before proceeding with credit calculation.

❻ Calculates credit using the results from parallel operations.

❼ Ensures the independent work completes before method returns.

We've used `AtomicReference` from the standard Java concurrency package—a thread-safe data structure that contains a reference to a single object. Since we are dealing with multiple threads, it is essential that we use it.

TIP

Use `AtomicReference` when you need to share object references safely between threads. For primitive values, consider using `AtomicInteger`, `AtomicLong`, or other atomic primitives.

To complete our credit calculation service, we need to implement the supporting methods. Each method simulates real-world operations like database queries and API calls:

```
// Simulated methods with 200ms delay each
private Person getPerson(Long personId) {
    simulateDelay(200);
    return new Person(personId, "John Doe");
}

private List<Asset> getAssets(Person person) {
    simulateDelay(200);
    return List.of(
        new Asset("House", 300000),
        new Asset("Car", 25000)
    );
}

private List<Liability> getLiabilities(Person person) {
    simulateDelay(200);
    return List.of(
        new Liability("Mortgage", 200000),
        new Liability("Credit Card", 5000)
    );
}

private void importantWork() {
    simulateDelay(200);
    System.out.println("Important work completed");
}
```

```
    private Credit calculateCredits(List<Asset> assets,
                                    List<Liability> liabilities) {
            simulateDelay(200);
            double totalAssets = assets.stream().mapToDouble(Asset::value).sum
            double totalLiabilities = liabilities.stream()
                .mapToDouble(Liability::amount)
                .sum();
            double creditScore = (totalAssets - totalLiabilities) / 1000;
            return new Credit(creditScore);
        }


    private void simulateDelay(int milliseconds) {
        try {
            Thread.sleep(milliseconds);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            throw new RuntimeException(e);
        }
    }
```

HANDLING INTERRUPTEDEXCEPTION CORRECTLY

When dealing with `InterruptedException` in Java, it's crucial to preserve the thread's interrupted status. The pattern shown demonstrates the proper way to handle this exception:

```
catch (InterruptedException e) {
    Thread.currentThread().interrupt();
    throw new RuntimeException(e);
}
```

When a thread is interrupted via `Thread.interrupt()` sets an internal flag, causing blocking methods (such as `sleep()`, `wait()`, or blocking I/O operations) to throw `InterruptedException`. However, catching this exception clears the interrupted flag, which can cause problems for code higher up the call stack that needs to know about the interruption.

`Thread.currentThread().interrupt()` resets the interrupted flag on the current thread. This ensures that any calling code checking `Thread.interrupted()` or `Thread.currentThread().isInterrupted()` will correctly see that an interruption occurred.

We can wrap the `InterruptedException` in a `RuntimeException`, which then allows us the interruption to propagate up the call stack even through methods that don't declare `InterruptedException` in their throws clause. This is particularly useful in contexts where you cannot add checked exceptions to method signatures (such as when implementing interfaces that don't declare them).

For comparison, we will also keep our original sequential version in the code:

```
// Sequential version (original)
public Credit calculateCredit(Long personId) {
// Method body available in the above.
}
```

To analyze performance improvements, it's helpful to measure the execution time of methods. Here's a utility class for this purpose:

```
import java.util.concurrent.Callable;

public class ExecutionTimer {
    public static <T> T measure(Callable<T> task) throws Exception {
        long startTime = System.nanoTime();
        try {
            return task.call();
        } finally {
            long endTime = System.nanoTime();
            // Convert to milliseconds
            long duration = (endTime - startTime) / 1_000_000;
            System.out.println("Execution time: " + duration + " milliseco
        }
    }
}
```

TIP

This is a simplistic approach for understanding code execution. For robust benchmarking, consider using a microbenchmark harness (like JMH) to account for factors like JVM warmup, dead-code elimination, and other optimizations.

Assuming that each method has a 200-millisecond execution time, we can determine how much time the preceding code will require for completion.

The `Thread.sleep(200)` method can be used to simulate this execution time. Now, let's create a demonstration that compares both approaches:

```
public class ParallelExecutionDemo {

    public static void main(String[] args) throws Exception {
        CreditCalculatorService service = new CreditCalculatorService();

        System.out.println("=== Sequential Execution ===");
        ExecutionTimer.measure(() -> service.calculateCredit(1L));
```

```
            System.out.println("\n=== Parallel Execution ===");
            ExecutionTimer.measure(() -> {
                try {
                    return service.calculateCreditWithUnboundedThreads(1L);
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                    throw new RuntimeException(e);
                }
            });
        }
    }
```

If we execute the preceding code, it will print something like this:

```
=== Sequential Execution ===
Important work completed
Execution time: 1026 milliseconds
=== Parallel Execution ===
Important work completed
Execution time: 616 milliseconds
```

We've significantly improved the performance by refactoring the credit calculation and employing multiple threads. The new version runs 1.66 times faster than the old one (616 milliseconds compared to 1,026 milliseconds for the previous version), which is noticeably faster for the user.

Though parallelization enhances performance by mitigating or reducing the blocking time of the initiating thread, it comes with its own challenges, particularly concerning thread management and system resource utilization. The ad hoc creation of threads, as demonstrated, lacks a controlled mechanism for managing thread lifecycles and resource allocation. This can lead to an overproliferation of threads, which has the potential to exhaust system resources. Each thread, being a heavyweight resource, consumes memory and processing power. As a result, since threads are created on an ad hoc basis and there is insufficient control, threads will be created in abundance, leading to `java.lang.OutOfMemoryError` exceptions, crashes, and other runtime bottlenecks.

Introducing the Executor Framework

To avoid the dangers of creating ad hoc threads, it's better to use one of the Java concurrency frameworks, such as the `ExecutorService`, that provides a more structured way of working with threads.

Furthermore, using an `ExecutorService` not only controls the number of concurrent running threads but also allows for efficient reuse of threads by managing their lifecycle for us, thus overcoming the overhead brought by the thread lifecycle.

Let's refactor the previous code and use `ExecutorService`:

```
public Credit calculateCreditWithExecutor(Long personId)
    throws ExecutionException, InterruptedException {

    try (ExecutorService executor = Executors.newFixedThreadPool(5)) { ❶
      var person = getPerson(personId);

      var assetsFuture = executor.submit(() -> getAssets(person)); ❷
      var liabilitiesFuture = executor.submit(() -> getLiabilities(person)
      executor.submit(() -> importantWork()); ❸

      return calculateCredits(assetsFuture.get(), liabilitiesFuture.get())
    }
}
```

Let's walk through what we have done here:

❶ Creates a thread pool that will be automatically shut down when the `try` block exits

❷ Submits both critical tasks that we need results from

❸ Submits additional work that doesn't need to block the main flow

❹ Waits for the critical tasks and processes their results

Now, let's measure the execution time of the code using `ExecutionTimer.measure()` and compare the results to the previous implementation:

```
public static void main(String[] args) throws Exception {
  System.out.println("\n=== Parallel Execution With Executors ===");
  ExecutionTimer.measure(() -> {
    try {
      return service.calculateCreditWithExecutor(1L);
    } catch (InterruptedException e) {
      Thread.currentThread().interrupt();
      throw new RuntimeException(e);
    }
  });
}
```

If we run the code, we will get output as follows:

```
=== Parallel Execution With Executors ===
Important work completed
Execution time: 630 milliseconds
```

Interestingly, the execution time remains very similar to the previous version. The executors provide useful facilities for managing concurrent work in Java applications—they make it easier to create and manage threads, they provide possible speedups for suitable workloads, and they make the code easier to organize and keep clean. Now, let's address the outstanding obstacles of the Executors package.

Remaining Challenges

The Executor framework brings significant improvements in resource management and asynchronous execution to Java applications. However, to maximize its benefits, it's crucial to be aware of its limitations.

Blocking on `Future.get()`

Despite introducing asynchrony via `Future` objects, the `get()` method call still blocks execution. This means that while we've shifted the blocking from one place to another, we haven't completely eliminated it.

Potential for cache coherence overhead and false sharing

When tasks are submitted to a thread pool, they are executed by threads that may run on different CPU cores than the main thread. This could lead to scenarios where the cache states of the two cores become inconsistent, creating the potential for performance degradation due to cache coherence protocols constantly invalidating and synchronizing cache lines across cores.

The idea is to improve data access speed so all modern CPUs have L-caches.[3] These caches store small chunks of data (size varies with hardware architecture) so that the next time the CPU needs that data, it can be retrieved from the cache instead of from slower main memory. This chunk of data is called a *cache line*, and its use significantly improves overall performance.

However, when multiple tasks run concurrently, there's a chance that data from two subsequent tasks might reside in the same cache line. If both tasks run on the same CPU, there's no need to look up the main memory again, as the data is already cached. That means the second operation will have been faster than the first, which is a small improvement. However, if the two tasks run on different CPUs, the second CPU has to invalidate its cache line if the

first CPU modified any part of it, then reload the entire cache line from main memory or another core's cache.

This phenomenon, known as false sharing, occurs when different threads modify different variables that happen to share the same cache line. The frequent cache line invalidations and reloads may negatively impact overall performance in frameworks like the Executor framework, where tasks are distributed across CPUs.

Lack of composability

The Executor framework is still quite imperative in nature. There is nothing wrong with imperative-style code, but many developers find functional and declarative styles to be easier to read and maintain.

CACHE COHERENCE PROTOCOLS

Modern CPUs use cache coherence protocols (e.g., MESI, MOESI) to ensure all cores see a consistent view of memory. When one core writes to a cache line, the protocol invalidates or updates that line in other cores' caches. This guarantees correctness but can introduce performance overhead due to frequent invalidations and data transfers across cores.

This leads back to our starting point: the natural question is "What now?" How do we solve these problems, and how can we use executors to write even more efficient and maintainable code in the future?

Beyond Basic Thread Pools

The Executor framework is very helpful; however, scenarios like heavy cache contention or the need for complex task dependencies can be problematic, as I stated previously. Java's Fork/Join Pool addresses these challenges with a performance-focused design and specialized algorithms. This dedicated implementation of `ExecutorService` delivers a more sophisticated thread-pooling mechanism, enhancing performance and resource efficiency through intelligent task scheduling. Let's explore further.

Cache Affinity and Task Distribution

Cache affinity refers to the concept of completing tasks in a way that leverages the locality of the CPU cache. If a task is executed on a particular CPU, its data gets loaded into the cache of that core. If other related tasks are executed on the same core, those other tasks can benefit from

the data already cached there, reducing the memory access time and, thus, improving performance.

The Fork/Join pool encourages cache affinity because worker threads usually execute tasks from their own local queues. This is especially helpful if the tasks frequently access the same shared data. By maintaining cache affinity, the Fork/Join Pool minimizes cache misses, which occur when the data needed by a task is not present in the cache and must be fetched from the main memory—a significantly slower process.

Consider, for instance, a set of parallel tasks working on a large array. If these tasks are all processing different slices of that array, it pays to have them run on the same core; then, the data in those slices of an array can stay in the cache and be accessed quickly by the next task. Otherwise, we will have to continually reload this data into the cache. Cache affinity reduces the overhead associated with repeatedly loading data into the cache.

Work-Stealing Algorithm

The Fork/Join Pool also employs a dynamic work-stealing algorithm to efficiently balance the load among threads. Instead of a single shared task queue, each thread in a Fork/Join Pool has its own queue. When a thread runs out of work, it "steals" tasks from the tail of another thread's queue. This dynamic work-stealing algorithm maximizes CPU usage, ensuring that no thread sits idle while there are tasks to be done. Figure 1-3 illustrates the work-stealing algorithm used in the Fork/Join Pool. This method of dynamically redistributing tasks ensures that all threads remain productive, enhancing overall CPU utilization and minimizing idle time.

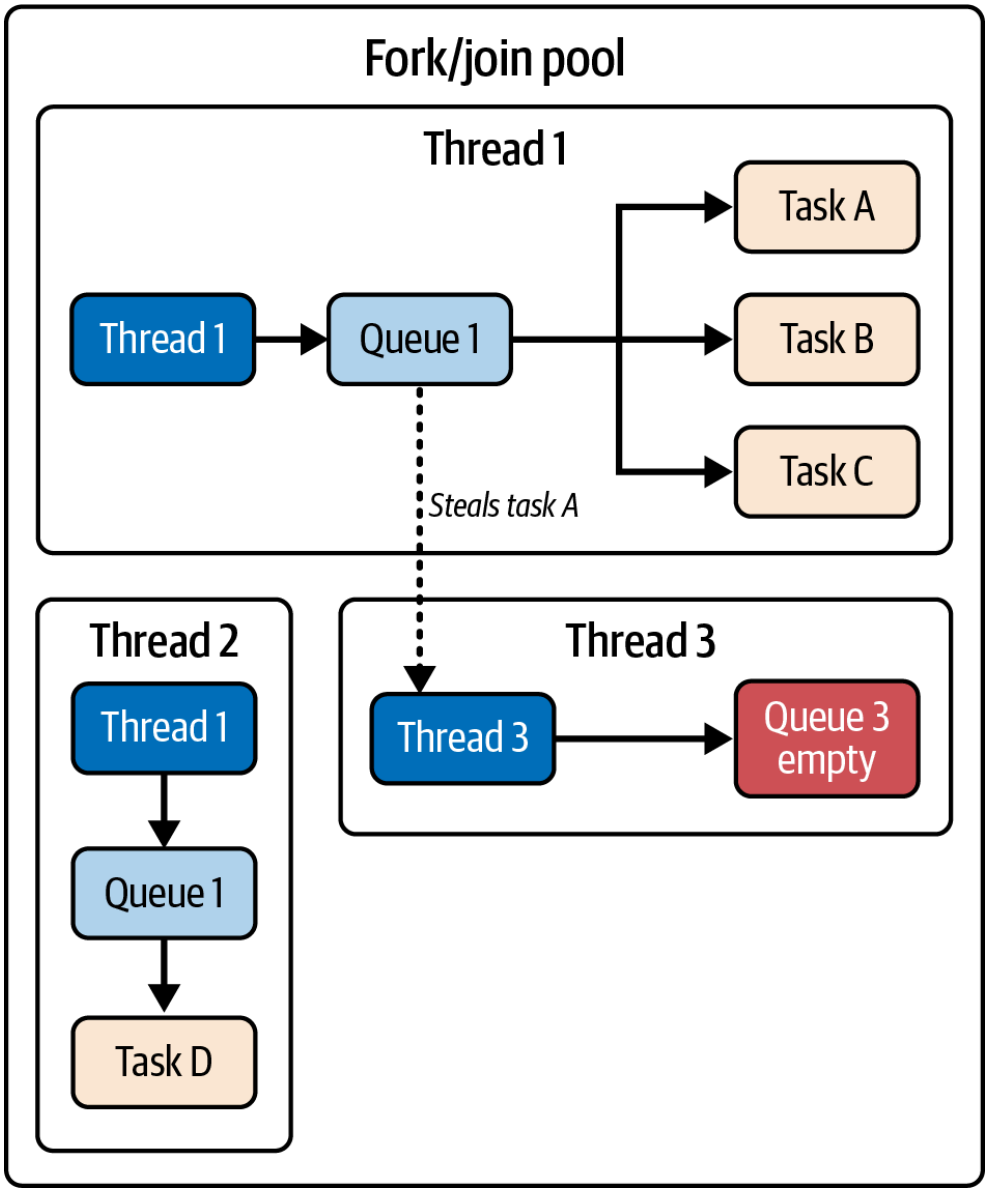

Figure 1-3. Work-stealing algorithm in Fork/Join Pool

Example of using a Fork/Join Pool

The Fork/Join Pool simplifies the process of sending tasks, and it automatically distributes the tasks efficiently without requiring additional management. Here's a basic example of how to use the Fork/Join Pool:

```
ForkJoinPool forkJoinPool = new ForkJoinPool();
forkJoinPool.submit(() -> {
    // your parallelized tasks here
}).join();
```

Notice how easy it is to submit tasks to a Fork/Join Pool. The framework automatically handles efficient work distribution, taking the burden of complex thread management off the developer.

Typically, the Fork/Join Pool is associated with the divide-and-conquer algorithm, using the `RecursiveAction` and `RecursiveTask` classes to break tasks into smaller pieces for the Fork/Join Pool. However, in this

context, we are focusing on the Fork/Join Pool itself, not the divide-and-conquer strategies used to create these tasks.

I'll discuss the Fork/Join Pool in more detail in the following chapter.

Bringing Composability into Play with CompletableFuture

The Executor framework and the Fork/Join Pool offer performance improvements, but composing complex asynchronous workflows can still be cumbersome. To address this challenge, Java 8 introduced `CompletableFuture`, a class designed to streamline asynchronous programming by focusing on composability and ease of use.

Let's look at how a composable and fluent API with `CompletableFuture` aligns with the earlier example.

Composable and fluent API

`CompletableFuture` transforms how we write asynchronous code, moving away from callback-heavy styles toward a declarative and functional approach. Its rich API empowers us to chain multiple asynchronous operations effortlessly, improving code readability and maintainability. Let's revisit the previous example:

```
import java.util.concurrent.CompletableFuture;
import java.util.concurrent.ExecutionException;
import static java.util.concurrent.CompletableFuture.*;

public Credit calculateCreditWithCompletableFuture(Long personId)
        throws InterruptedException, ExecutionException {
    return runAsync(() -> importantWork()) ❶
        .thenCompose(aVoid -> supplyAsync(() -> getPerson(personId))) ❷
        .thenCombineAsync(supplyAsync(() -> getAssets(getPerson(personId))),
                (person, assets)
                    -> calculateCredits(assets, getLiabilities(person))) ❸❹
        .get(); ❺
}
```

Let's examine how this `CompletableFuture` approach works:

❶ Starts the independent work asynchronously

❷ After important work completes, begins fetching `person` data asynchronously

❸ Combines the `person` data with `assets` fetched in parallel

❹ Calculates the final credit score using the combined results

❺ Blocks and waits for the final result to complete

This example demonstrates how `CompletableFuture` allows us to structure asynchronous workflows in a clear and concise manner. However, it is not without its disadvantages.

Let's explore the advantages and disadvantages of this API.

Advantages of using CompletableFuture

`CompletableFuture` transforms how you structure asynchronous code with its fluent API. Instead of tangled nested callbacks or sprawling class hierarchies of task objects, you define workflows as a series of transformations and intermediate computations. This improves readability and maintainability, reducing the likelihood of bugs in applications where asynchrony is central.

Under the hood, `CompletableFuture` builds on the Fork/Join framework, leveraging optimized work-stealing for efficient task distribution and better scalability under load. Built-in methods such as exceptionally, handle, and `whenComplete` give you explicit control over error handling. You can also supply custom executors to fine-tune thread management for specific workloads. Finally, while a blocking `get()` is sometimes required at boundaries, the majority of your pipeline can remain non-blocking, enabling highly responsive and resource-efficient code.

Disadvantages and limitations

Although `CompletableFuture` is a powerful feature in Java, it does require you to shift your mindset if you are already used to the imperative programming style. I would say that you may have to invest quite a bit to learn this rich API in order to use it in a meaningful way. You have to figure out the best places or value propagation techniques for the various methods it offers. This would be a considerable investment of time and practice, and with that comes the possibility of failing to use it correctly.

For example, the `get()` method is still a blocking method, which is necessary but can induce blocking in a flow that would otherwise undermine the use of asynchronous programming altogether if not used frugally. You also can have an issue in multichain `CompletableFuture` cases where you have more than one dependency and have to propagate and check the error in order to recover. This, if not designed correctly, can create a very big nightmare to debug. Finally, debugging code written in `CompletableFuture`'s chained style poses another challenge in flow control. If you are used to the default debugging tools available in various IDEs, stepping through the code, setting a breakpoint at a particular spot,

and seeing the context of a statement by taking a step backward in the code is going to be highly challenging for you. This is because of the inverted and ambiguous execution flow of asynchronous code—it doesn't just get executed line by line.

NOTE

While the benefits of asynchronous programming are clear, and `CompletableFuture` provides powerful features along with other alternatives like reactive frameworks, we must ask a critical question: Are we organizationally ready for this complexity?

Asynchronous programming fundamentally changes how we design, debug, and maintain applications. The gains in performance come with significant costs in cognitive overhead, debugging difficulty, and architectural complexity. Before adopting these patterns, consider whether your team has the experience to maintain clean code principles, handle complex error scenarios, and debug asynchronous call chains effectively.

This isn't merely a technical decision. It's an architectural commitment that affects every aspect of your application's lifecycle. Choose wisely based on your team's capabilities and your application's actual performance requirements.

A Different Paradigm for Asynchronous Programming

Although `CompletableFuture` provides powerful tools, changing one's mindset is not enough to address the remaining limitations (or to reach the higher levels of performance achievable in some cases). Reactive programming introduces a paradigm focused on data streams, asynchronous event processing, and non-blocking operations. Frameworks like RxJava, Akka, Eclipse Vert.x, Spring WebFlux, and others implement this paradigm in Java with rich toolsets.

For instance, let's reimagine our previous `calculateCredit()` example using Spring WebFlux. First, we need to add the necessary dependency and imports:

```
<dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-webflux</artifactId>
    <version>6.0.0</version>
</dependency>
```

TIP

To test the reactive example, create a Maven project and add the `reactor-core` dependency, as shown in the preceding *pom.xml*.

Now we'll add the following method to our `CreditCalculatorService.java` class:

```
public Mono<Credit> calculateCreditReactive(Long personId) {
    Mono<Void> importantWorkMono = Mono.fromRunnable(() -> importantWork()
    Mono<Person> personMono = Mono.fromSupplier(() -> getPerson(personId))
    Mono<List<Asset>> assetsMono = personMono
        .map(person -> getAssets(person)); ❸
    Mono<List<Liability>> liabilitiesMono = personMono
        .map(person -> getLiabilities(person)); ❹

    return importantWorkMono.then( ❺
        Mono.zip(assetsMono, liabilitiesMono) ❻
            .map(tuple -> {
              List<Asset> assets = tuple.getT1();
              List<Liability> liabilities = tuple.getT2();
              return calculateCredits(assets, liabilities); ❼
            })
    );
  }
```

Let's examine the reactive flow:

❶ Creates a `Mono` that executes important work asynchronously when subscribed

❷ Wraps the `person` lookup in a reactive stream

❸ Transforms the `person` stream to fetch assets asynchronously

❹ Transforms the `person` stream to fetch liabilities asynchronously

❺ Waits for important work to complete, then proceeds with credit calculation

❻ Combines `assets` and `liabilities` streams into a single stream containing both results

❼ Calculates the final credit score from the combined data

Looking at this code, you'll notice it's conceptually similar to the `CompletableFuture` approach but uses reactive streams terminology and operators. However, if you're unfamiliar with reactive programming, this approach can be puzzling and requires significant learning investment to understand concepts like Publishers, Subscribers, backpressure, and reactive operators.

I will not explain reactive programming in this book, as this is not the goal of this book, but you can explore other books written about the reactive stack.

Let's discuss some of the limitations we encountered in the reactive programming approach just to make sense of why we need something different or why we need virtual threads, which *is* the goal of this book.

Drawbacks of Using Reactive Frameworks

As the saying goes, "There's no such thing as a free lunch," and the same applies to reactive programming. Let's outline some of the key challenges it presents:

Steep learning curve

: Grasping the fundamentals of the reactive programming paradigm requires a significant mental shift for developers who are already used to the imperative programming paradigm. Concepts like Observables, Observable operators, schedulers, and backpressure, which are fundamental to reactive frameworks, require an investment of time and a potentially steep learning curve compared to other concurrency models.

Increased cognitive load

: The functional programming style used in reactive frameworks can be too much for full-time developers with long tenures of imperative programming or object-oriented programming under their belts. Along with the heavy usage of lambdas, higher-order functions, and functional composition, chains of operators and transformations can make the code itself harder to grasp initially and, thus, more challenging to maintain (at least in the short term) among large projects or teams where not all the members have a similar amount of reactive programming experience.

Debugging difficulties

: Traditional debugging tools and techniques don't always work so well with reactive systems on the fly. For instance, chained operators lead to asynchronous execution where, if an error does arise, the stack trace might provide insufficient context to find the root of the problem. You might miss the event that last occurred through event hopping between threads and operators. Specialized debugging tools, possibly provided as part of your reactive framework, might be required. This adds to the learning curve. For example, suppose a reactive pipeline fetches data from four different sources, transforms the four streams, and then combines the results. An error arising from the combine phase might result in a stack trace that points to the combine operator and makes it difficult to tell whether a particular upstream source or

transformation is the root cause of the problem. Tracking the data flow throughout the reactive pipeline becomes tricky in complex pipelines with dozens of operators, tens of asynchronous operations, and so on. In the traditional threading model, debugging is relatively straightforward and comes at little cost.

Overcomplication risk

The composability possible through reactive frameworks' operator-based models, while very powerful, comes with the potential to create more complex products than are absolutely necessary for a given business requirement or user interface element. Just as one might be able to cook a chicken curry using every dish in a family's silver set, it is possible (and tempting) to solve every problem with something slightly more complicated than is really necessary. As such, reactive frameworks face the creative risk of allowing inexperienced programmers to overengineer their domain—a risk not unique to reactive frameworks themselves but one that occurs in any powerful abstraction or library when used by those who do not know when to stop.

Potential mismatch

Reactive programming frameworks are best suited for scenarios involving significant asynchronous operations, event-driven data flows, or high-throughput data streams. In other situations, some subset of reactive systems' or frameworks' capabilities might not be needed because there isn't much asynchronicity or the data flow isn't inherently event-oriented, or the effort to learn, abstract, and apply into the framework justifies the greater simplicity of a more basic synchronous or request-response approach.

Vendor lock-in

The essential ideas of reactive programming are undoubtedly transferable; however, each of the significant reactive frameworks has its own set of nuanced APIs. Picking one of them over the other means you will be locked into a specific framework, which naturally makes it harder to change libraries later, and potentially constrains flexibility somewhere down the line of the project's lifecycle. This lock-in can also have implications, such as difficulty in finding developers familiar with the chosen framework, potential for framework stagnation or lack of continued support, and challenges in migrating to a different framework if needed.

Having mentioned all of these challenges, there is absolutely no denying that reactive programming frameworks have the potential to enormously simplify the management of sophisticated asynchronous scenarios. However, we must examine their trade-offs in detail to understand if the benefits outweigh the costs for the project we're working on.

Revolutionizing Concurrency in Java

Java's traditional concurrency mechanisms have served us well, but as the application of concurrency has grown to more complex and additional use cases, various challenges have been presented along the way. That's why it is essential to explore new solutions.

The Promise of Virtual Threads

Now that we understand the shortcomings of our traditional approaches, we need to find a better way to fix those problems. This is precisely where Project Loom, a key step toward first-class modern concurrency in Java, comes into play. It introduced a new kind of thread called *virtual threads*, ushering in a new era of concurrency in Java. These are very lightweight threads and can be created on demand with next to no overhead compared to what is traditionally associated with thread creation. Virtual threads can be instantiated in the millions, unlocking new possibilities for highly concurrent and scalable applications.

Let's discuss some of the characteristics of virtual threads.

Seamless Integration with Existing Codebases

One of the key strengths of Project Loom lies in its seamless compatibility with existing Java codebases. For instance, if your application already uses the Executor framework, all you have to do to take advantage of virtual threads is pass an `Executors.newVirtualThreadPerTaskExecutor()` to your existing `ExecutorService`:

```
Credit calculateCreditWithVirtualThread
    (Long personId) throws ExecutionException, InterruptedException {
  try (var executor = Executors.newVirtualThreadPerTaskExecutor()) { ❶
    var person = getPerson(personId);
    var assetsFuture = executor.submit(() -> getAssets(person));
    var liabilitiesFuture = executor.submit(() -> getLiabilities(person)
    executor.submit(this::importantWork);
    return calculateCredits(assetsFuture.get(), liabilitiesFuture.get())
  }
}
```

❶ Replace any traditional executor with `Executors.newVirtualThreadPerTaskExecutor()` to immediately gain the benefits of virtual threads.

This ease of integration ensures a smooth transition to the new concurrency model.

Virtual Threads and Platform Threads

Virtual threads ride on top of *platform threads*, also known as *classical threads*, which are also backed by the Fork/Join Pool, so they inherit the benefits of that more sophisticated thread pool. Thus, virtual and platform threads combine perfectly to produce the best resource utilization and scalability results.

Intelligent Handling of Blocking Operations

One of the most exciting aspects of virtual threads is their intelligent handling of blocking operations. When a virtual thread encounters a blocking operation—such as a sleep or network I/O—it automatically yields control back to the underlying platform thread. This allows the platform thread to continue executing other virtual threads, ensuring optimal utilization of available resources. Once the blocking operation is complete, the virtual thread can simply take up execution from where it left off, avoiding perceptible bottlenecks.

Benefits of Embracing Virtual Threads

Before diving into the nitty-gritty of virtual threads in the next chapter, let's take a quick look at how virtual threads revolutionize the way we handle concurrency in Java. Here are some of the key advantages:

Resource efficiency

: Virtual threads are lightweight, and we can spawn millions without exhausting resources, which permits highly concurrent, scalable applications.

Code simplicity

: With the ability to write blocking code without performance penalties, developers can now write and maintain a more straightforward, traditional imperative coding style. That means no additional learning curve and easier-to-maintain code.

Optimal utilization

: During blocking operations, virtual threads relinquish control, which ensures that platform threads remain utilized, increasing CPU and application performance.

One of the biggest concerns for developers using concurrency in Java is the specific information they must learn to use it correctly. Project Loom

represents a huge step forward for the concurrent style Java developers create and use; it will let them write highly concurrent and efficient applications at scale while still using the patterns they are already familiar with.

In Closing

In this book, we will continue discussing virtual threads and explore more about this revolutionary technology in the next chapter.

As you dive deeper into virtual thread concepts, consider how they can transform your approach to designing concurrent applications. Virtual threads offer not just a new way to handle concurrency but also a more efficient, scalable, and intuitive method that aligns with modern computing demands.

[1] Little's Law is a key principle in queuing systems, including multithreaded applications. It relates latency, concurrency, and throughput, which are crucial for computing performance. In the next chapter, we'll discuss this in detail and demonstrate how higher concurrency leads to higher throughput.

[2] This is the default size in most Linux environments, but it depends on the operating system and can be tweaked.

[3] L-caches are the tiered levels of superfast memory (L1, L2, L3) within a CPU that store frequently used data for quick access.